Lab 1: blinky disco on bare metal
a.k.a., intro to PlatformIO, VScode, GPIO and blinking an LED

What we will learn in this lab

- How to use our IDE, which is PlatformIO running on VScode. We'll be writing a simple program on the host, cross-compiling on the host, flashing our Nucleo board and running the code.
- Programming a GPIO pin and using it to blink an LED on and off. We'll do it two ways: first with the Arduino API (which we'll implement ourselves by directly programming CSRs). Second, using the STM32 HAL API. Using HAL may be new to most people, but it's a useful job skill :-).
- The shortcomings of programming with a single thread; blinking one LED will be easy, but blinking two LEDs will not be.

Setting up the lab: tour of the 432 Nucleo board.

Start out by grabbing a 432 Nucleo board. Can you identify

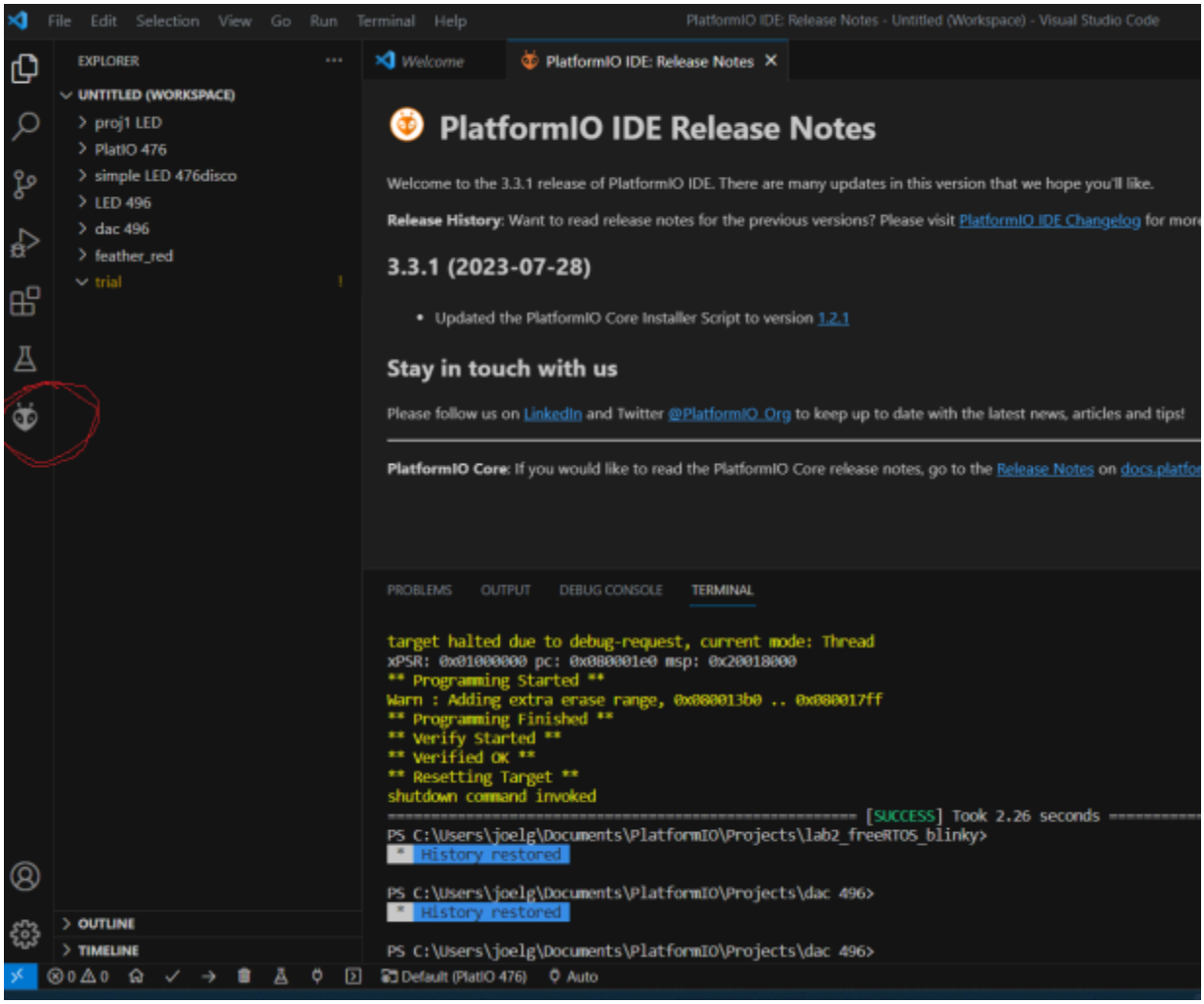
- The processor (STM32L432). Hint for reading really small letters: use your phone camera, and then zoom in.
- USB MicroB port for host communications.
- Green LED (good luck on this! You probably won't succeed until later).

Don't plug the Nucleo board into your PC quite yet; we'll do that later.

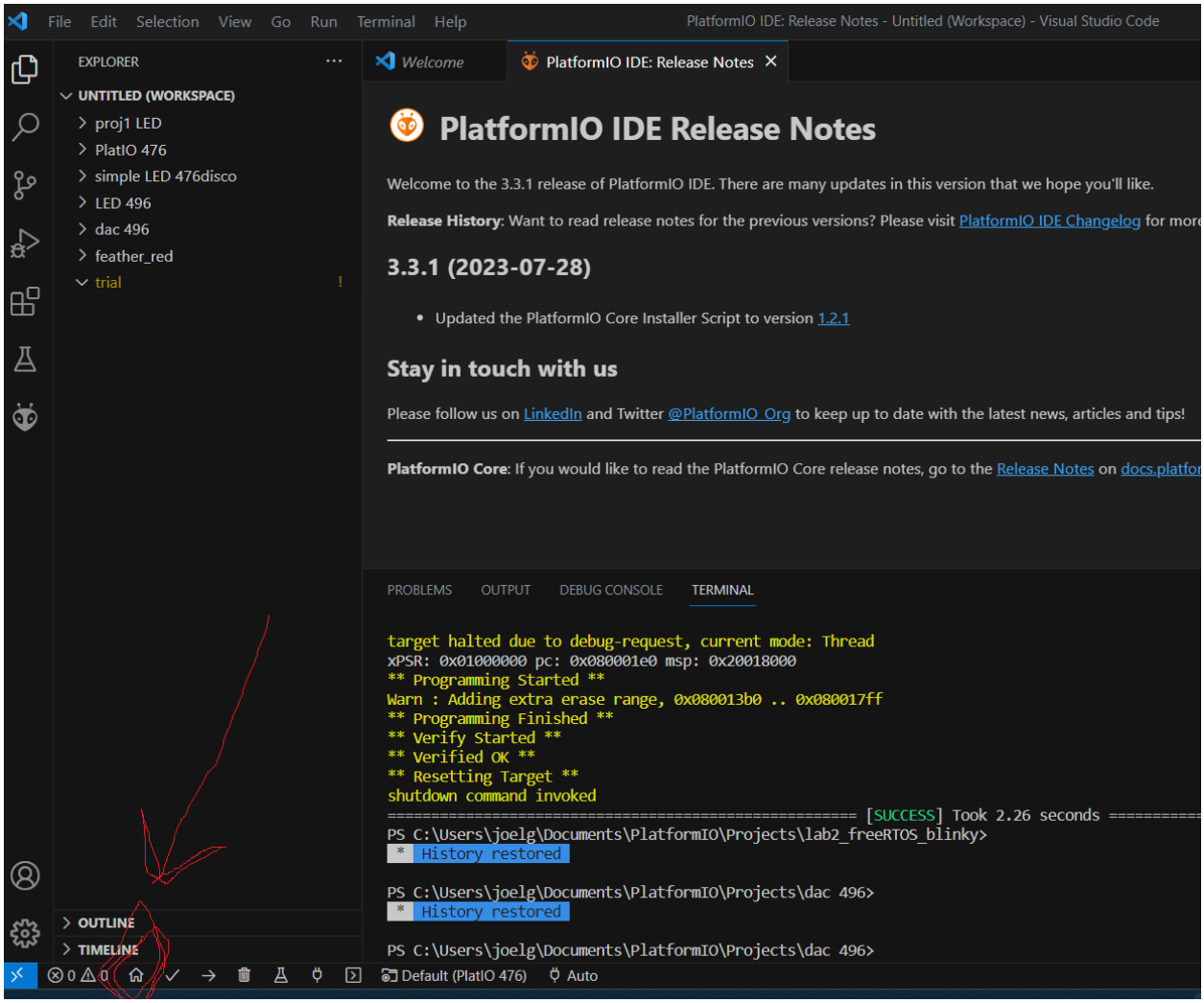
PlatformIO and VScode: building a project

Here are directions and pictures to build your project in VSCode.
Start off by running VSCode. When you do so, VSCode may or may not come up in PlatformIO mode, looking mostly like the picture below. If not, then the way to get it into PlatformIO mode is to click the "ant"-face on the left.

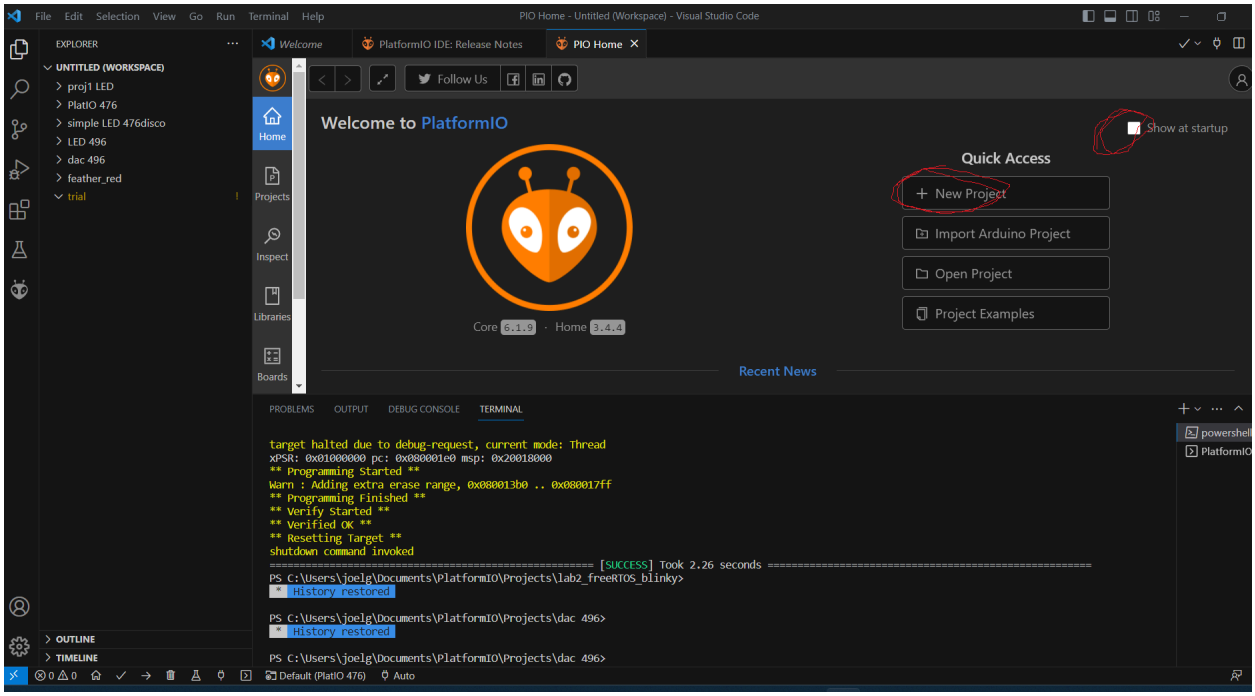
When you launch VSCode/PlatformIO for the first time on a new computer, it may ask you to wait for several minutes while it fetches collateral files from the web, as well as asking you to exit and re-enter VSCode. All of this is normal: look for messages like "PlatformIO Installer: Installing PlatformIO Core" or "Installing PlatformIO IDE… It may take a few minutes depending on your connection speed. Please do not close this window until this process is completed."



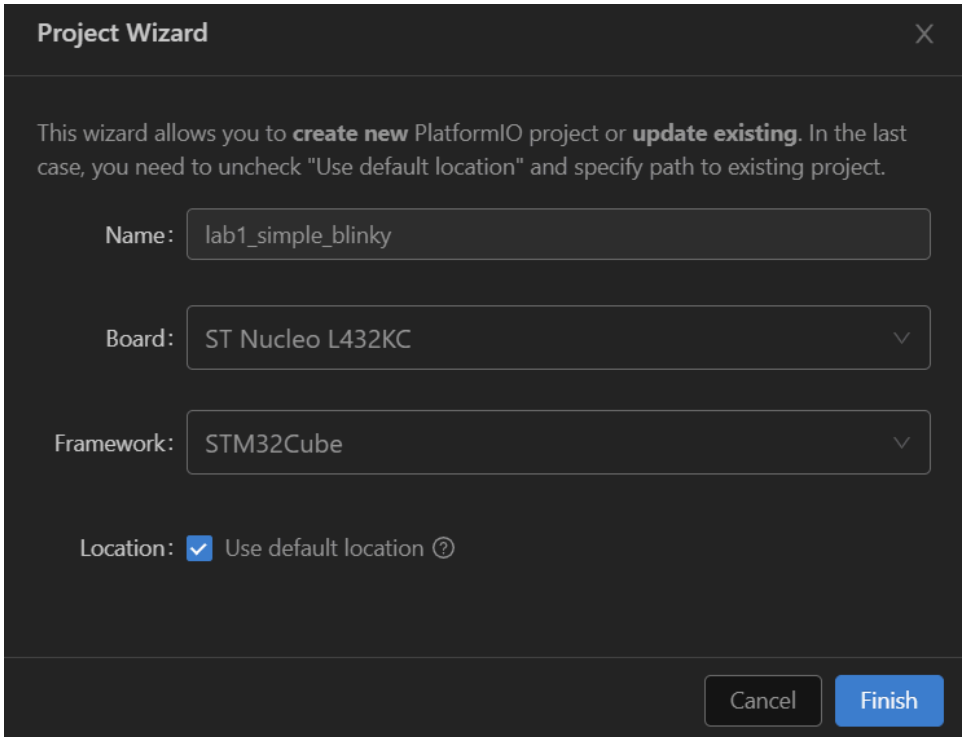
Next, we'll build a new project. Start off by getting to the PlatformIO home screen by clicking the "home" icon on the bottom left as shown below (again, you may have to exit and enter VSCode for the menus to set up the very first time):



You will now have the PlatformIO home window taking up most of the top half of the screen. Click on “+ New Project” as shown below. If you like, you can also select the “Show at Startup” check-box so that you can skip these steps when building new projects in the future.



With that done, a pop-up *Project Wizard* window will appear (see picture below). Name your project *lab1_bare_metal_blinky*. The choice of boards can be annoying: there are well over a thousand choices, and scrolling to find our board can be slow. The trick is to type “432” into the board slot, and PlatformIO will then restrict the choices to only a few. Pick “ST Nucleo L432KC.” Finally, use STM32Cube as the framework:



Why did we choose “STM32Cube” as our framework? This tells PlatformIO which library .c and .h files to use. If we were only programming CSR bits, we could have chosen the simpler framework “CMSIS,” but “STM32Cube” gives us access to the full HAL API.

Click on “Finish” to create the project. Note that sometimes you may have to scroll the popup to see the “Finish” button (depending on your screen size and the relative sizes of the top and bottom parts of the screen).

Congratulations! You now have a PlatformIO project. Now let’s look at the directory structure that platformIO has created for you.

What are the files and folders in your project?

All of your PlatformIO projects are in the directory tree starting at *C:\Users\your-user-name\Documents\PlatformIO\Projects*. Every new project you make will be one subdirectory of *Projects*. For now there should be only your one newly-created subdirectory, called *lab1_bare_metal_blinky*.

All the files for this particular project go under *lab1_bare_metal_blinky*. For today, we care most about the directories *src* (which includes your .c files) and *include* (for your .h files). Copy the files *lab1_main.c*, *lib_clock.c* and *lib_gpio_lab1.c* into your *src* folder, and *lib_ee152.h* into your *include* folder. You can find those files on the course web page – note that while *lib_clock.c* and *lib_ee152.h* come from a central repository, *lab1_main.c* and *lib_gpio_lab1.c* are custom for this lab. PlatformIO knows which files are which based on the directories. Any files in *src* get compiled, and any files in *include* get included. So if you put a .h file into *src* or a .c file into *include*, it will be ignored.

Let’s look at *main_lab1.c*. You can open the file in your favorite editor (as long as it’s installed on our lab machines). You can also open it in VSCode itself, especially if you’re already familiar with the VSCode IDE. To do that, simply choose File/Open File from the top-left menu, and browse until you get to *lab1_main.c*.

There’s not a lot of code here! That’s good, since it’s your first project. There are two **#include** statements on top; *lib_ee152.h* is the standard library-include file for this course, and *stm32l432xx.h* is a file provided by STM. It mostly defines symbolic constants for all of the CSRs and the bit fields in those CSRs.

Main() is also nice and small, mostly because *lib_ee152.h* does all of the heavy lifting. Look at the code in *main*(). It blinks the green LED at 1 Hz. Make sure you understand how it works, given the Arduino API that we discussed in class.

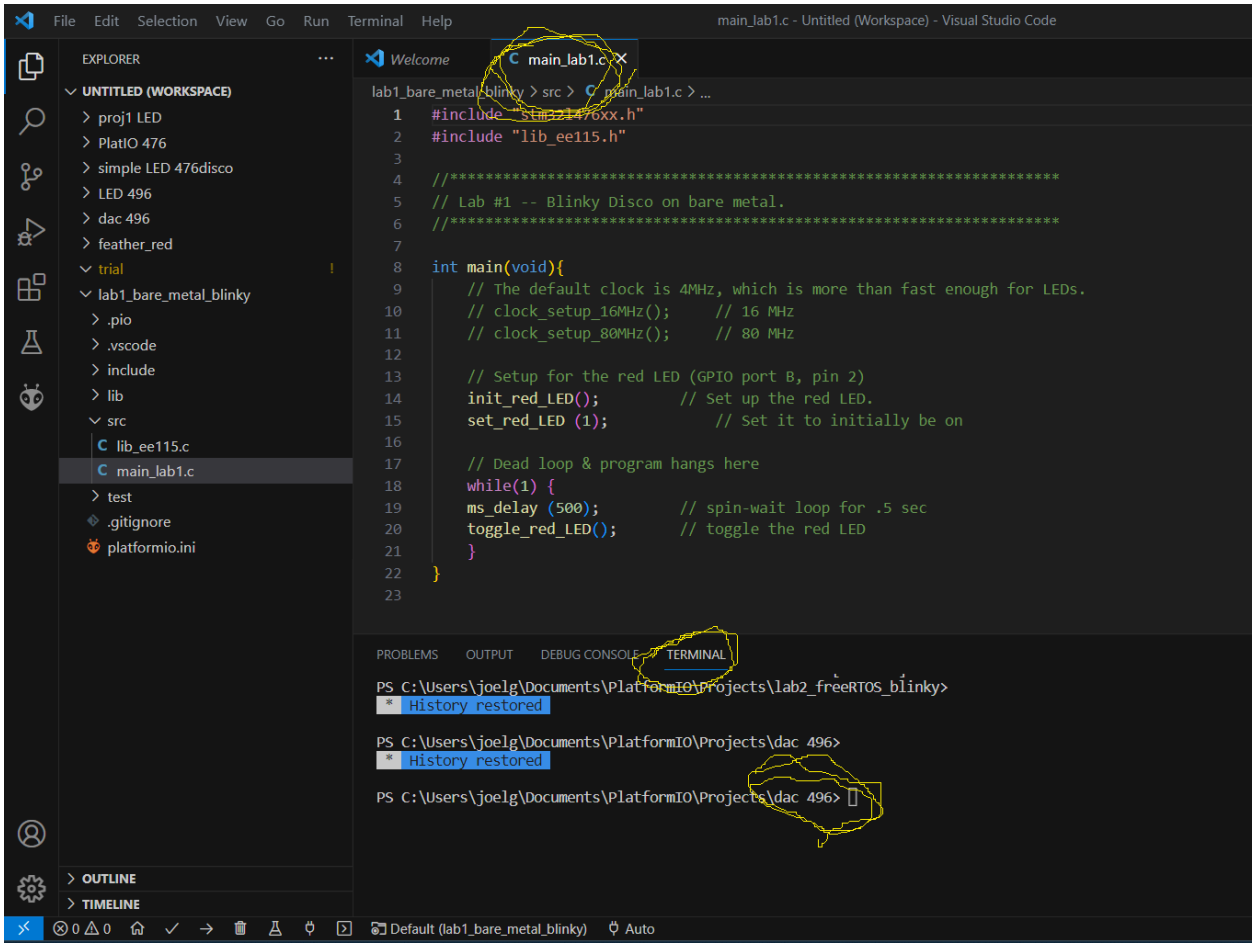
Fleshing out lib_ee152

Now it's time for the nitty-gritty: doing the CSR bit poking to implement our Arduino API. First look at *lib_ee152.h*. It declares (among many other functions) *pinMode*() and *digitalWrite*(), but doesn't actually implement either. Next, open up *lib_gpio.c* to find the skeleton implementations of those two functions. You will notice that there are two versions: with and without USE_HAL defined. For now, we're doing the "without," so that we are directly poking the CSRs.

Flesh out (again, in the not-USE_HAL version) *digitalWrite*() and *pinMode*() using the CSR fields that we discussed in class. Feel free to ask for help if needed. The file *stm32l432xx.h* is available on the course website for your reference.

Cross-compiling, flashing and running the project

PlatformIO makes it easy to cross-compile, flash and run your project. First, though, let's make sure you're in the right directory.



Your screen will probably look pretty similar to the screen shot above. Your *main_lab1.c* is on top. For now, let's look at the bottom of the screen. It's a *Terminal* window, and it acts a lot like a Linux shell. You'll notice that its command prompt is saying something a

bit disturbing: it's in the directory …\Projects\dac_496, which is an old project I happened to be working on some time ago. Yes, even though we just build a project, PlatformIO isn't always quite smart enough to realize that we probably want to work on that project now! Easy to fix, though: just type **cd ..\lab1_bare_metal_blinky** at the prompt, and problem fixed. As with most Linux shells, tab-completion works fine for file names.

Now let's get things running. Build the project (both cross-compiling and linking) with the check-mark symbol near the lower left (it's just to the right of the "home" icon you used before). You'll notice that in this case, we came up with a linker error for a multiply-defined symbol:

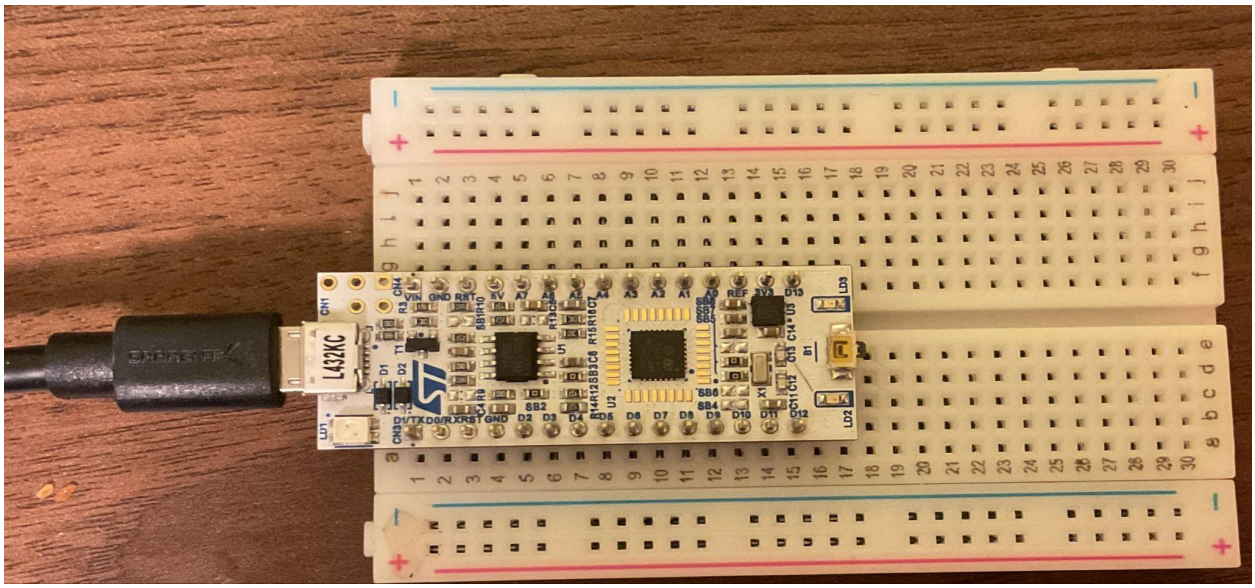
```
Linking .pio\build\disco_l476vg\firmware.elf
.pio\build\disco_l476vg\src\lib_ee115.o:(.data.SystemCoreClock+0x0): multiple definition of `SystemCoreClock'
.pio\build\disco_l476vg\FrameworkCMSIS\system_stm32l4xx.o:(.data.SystemCoreClock+0x0): first defined here
collect2.exe: error: ld returned 1 exit status
*** [.pio\build\disco_l476vg\firmware.elf] Error 1
========================================================== [FAILED] Took 2.75 seconds ==========================
```

This may or may not happen for you. If it does, the fix is simple: edit the file *platformio.ini* (which is in the directory *lab1_bare_metal_blinky*) as follows:

```
10
11  [env:nucleo_l432kc]
12  platform = ststm32
13  board = nucleo_l432kc
14  ;framework = cmsis
15  framework = stm32cube
16
17  build_flags =
18      -DLL_DEFINES_SYSTEMCORECLOCK
19
```

With that done (and remember to save the file!), the error should go away. Of course, the code you wrote yourself may have one or more compilation errors, which you would have to fix. Once you have a clean build, it's time to flash your Nucleo board. Don't hook up your Nucleo board to the PC quite yet. Instead, click on the right-arrow symbol (just to the right of the check-mark symbol that you just used to build the project). Surprise, surprise! You hopefully got an error message essentially saying it could not find any board to flash.

OK, now grab a USB cable (Micro-B to USB-A), and plug one end of the cable into the board:



Plug the other end of the cable into any USB port on the desktop, and now you're *really* ready to flash the board. Click on the right-arrow symbol again, and you should be fine.

Note that, as opposed to some other IDEs, in PlatformIO your code will start running as soon as it is flashed. So your green LED should now start blinking!

Also note that while you entered the build and flash commands graphically, PlatformIO also echoed in the Terminal window. So you can now use command recall in the Terminal window to re-issue any subsequent build or flash commands.

What if your code doesn't work?

If your code doesn't work, then it's time for debugging. What are some techniques? First, the tried-and-true simple ones:

- Double-check that you did not have any compiler or linker errors, and that any warnings are ignorable.
- Have another person look at your code.
- Have a TA look at your code.

The next step in debugging is to single-step with a debugger. The course website has a short video dedicated to that. It can let you see if your code is hanging somewhere, and even let you examine CSR fields.

Making code changes in main()

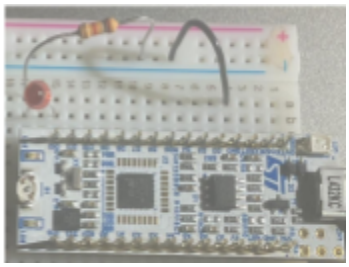
Once the code works, it's time to play with the code in *main*() a bit:

- Change the clock rate by commenting in or out the hopefully-evident lines. Perhaps surprisingly, this will not make any observable difference in the LED's flashing rate!
- Change the LED's flashing rate and the duty cycle.

Adding a red LED

For the software types among you, now you get to do some wiring. We're going to add a second LED: red, driven by Nano pin D12. Here's a schematic of the hookup:





Note that LEDs are directional: the two pins are *not* interchangeable, and if you insert the LED backwards it won't work. Two tricks: the positive end has a longer wire (unless somebody has trimmed it), and the negative end has a small flat notch on the side of the LED. And depending on what LED you find, the series resistor is usually between 100Ω and 1KΩ (we often have some cool LEDs with a built-in series resistor in our bins, too).

Blinking both LEDs at different frequencies

Now that you have two LEDs, you can blink them both. Nothing to it, right? First off, let's just blink both LEDs at the same time. This is hopefully pretty easy. You may run into bugs if your *pinMode*() and *digitalWrite*() functions are not properly handling the different pins.

The next step is to blink them alternately. So: red, green, red, green, red, green. Set them to alternate every half second. Hopefully this does not require a ton of ingenuity.

Your next challenge is, well, more challenging. Get them to blink at a two-on-three rhythm; i.e. the red LED goes through two on-off cycles every second, while the green LED goes through three on-off cycles every second.

Adding HAL

The final piece of this lab is to implement the same blinking, but now using HAL. Your big-picture plan is

1. Use CubeMX to generate the initialization code as per our demo in class
2. CubeMX will autogenerate the files *main.c* and *main.h* ; you should copy them into your PlatformIO project and rename them to *lab1_main_HAL.c* and *lab1_main_HAL.h*.
3. Add code to *lab1_main_HAL.c* to implement the 2-on-3 LED blinking. You should continue to use *delay*() as before to implement time delays, but now use the HAL function *HAL_GPIO_WritePin*() to write to a GPIO pin. The HAL reference manual is also available on the course website if you would like more information.

Here are some more details:

- When you build the new CubeMX project, our chip is an STM32L432KCU6.
- After you have set up the GPIO programming in CubeMX, use the "Clock Configuration" tab for clock programming. If you set the main FClk (CPU **F**ast **Cl**oc**k**) to 16 MHz, the rest of the tab should auto-populate. With this done, your *lab1_main_HAL.c* should contain all of the appropriate clock-setup code.
- In the "Project Manager/Project" tab, choose "Application structure"=Basic, "Toolchain/IDE"=Makefile. In the "Project Manager/Code Generator" tab, choose "Copy only the necessary library files," "Keep user code when regenerating," and "Delete previously-generated files when not regenerated." Under "Project Manager/Advanced Settings," make sure you have both RCC and GPIO set to "HAL" rather than "LL," and that "Do not generate function calls" is unchecked.
- CubeMX typically puts its generated code in the Windows folder *C:\Users\your-user-name\Documents\Cube\your_project_name*. The files that you need are *src\main.c* and *inc\main.h*. Copy *main.c* into your PlatformIO *src* directory, and *main.h* into your PlatformIO *inc* directory. To avoid confusion, please rename them as *lab1_main_HAL.c* and *lab1_main_HAL.h* (and you will of course have to change the "**#include** *main.h*" line in *lab1_main_HAL.c* accordingly).
- Reuse the existing Arduino-API *delay*() function that you've already used. Since the existing *lib_clock.c* is in the same PlatformIO folder as your new HAL code, this should work fine.
- There is an existing issue with HAL. Specifically, both *HAL_Init*() and *SystemClock_Config*() enable the STM32 SysTick interrupt, but HAL doesn't provide an interrupt handler for it. Thus, your program will by default run for about 1ms, then vector to a default interrupt handler, which merely runs in an infinite loop. To fix this, add the line "SysTick->CTRL &= ~SysTick_CTRL_TICKINT_Msk;" right after the call to *SystemClock_Config*() in *lab1_main_HAL.c*. If you're interested in a little bit of extra debugging, and you're a bit familiar with the STM interrupt system, feel free to (before adding this line), use the PlatformIO debugger to pause the code when it hangs and

confirm that it is in an infinite-loop default exception handler, and then look at the PSR to confirm that it is indeed in the SysTick handler.

In-lab checkout

Show one of the instructors that you

- got the red & green LEDs blinking at the same time
- got the 3-to-2 blink pattern working
- can explain your CMSIS code
- can explain your HAL code

What to turn in

1. Code snippets for *pinMode*(), *digitalWrite*() and the relevant parts of *lab1_main.c* and *lab1_main_HAL.c* as part of your lab report.
2. In a paragraph or two, explain your strategy for blinking two LEDs at different frequencies (referring to the relevant code snippets above), and explain how you might extend it to three or more LEDs at various frequencies. Is this what one might call a nice "modular" software solution?
3. Let's investigate a mystery. If you look inside the function *ms_delay*(), you'll see that it's implemented with a big loop of NOOPs. Doubling the clock speed should mean that the processor, execute roughly twice as many instructions per second, and should thus mean that *ms_delay*() executes roughly twice as fast. Nonetheless, you hopefully saw that changing the clock speed did not affect the LED blink rate at all. Look into *ms_delay*(), *clock_setup_16MHz*() and *clock_setup_80MHz*(); can you solve the mystery?

Each group should turn in just one lab report. And – note that this lab is due midnight BEFORE the day that we do lab #2, so that I can release the regular GPIO library for lab #2 after lab #1 is due.